# Mode d'emploi des scripts

## Cluster Zêta — Riemann_Lab

Aide-mémoire complet : à quoi sert chaque script, comment le lancer, sur quelle machine.

hprzeta • Projet Hypothèse de Riemann • 04/10/2026 (révision de l'édition du 19/09/2026) · version publiable : IP et domaine DDNS masqués

| Machine | Hôte / alias SSH | OS | Rôle |
|---|---|---|---|
| PC1 zeta-lab | local (<IP-PC1>) | Ubuntu 24.04 — 16 Go de RAM (depuis le 25/07/2026) | Orchestrateur, calcul principal, GPU |
| PC2 zeta-calc-second | <IP-PC2> | Debian 12 | Second nœud de calcul (Core2Duo) |
| PC3 zeta-backup | <IP-PC3> | Ubuntu 16.04 (Linux Lite 3.8, xenial, 32 bits) | Backup + monitoring (compte pjexosql) |
| PC4 zeta-secure | <IP-BASTION-WG> / <IP-BASTION-LAN> | OpenBSD 7.9 | Bastion VPN WireGuard |
| PC5 zeta-monitor | <IP-PC5> | Debian 12 | Monitoring |

# 1. Vue d'ensemble

Tous les scripts sont dans **~/projet_zeta/scripts/** sur PC1, sauf ceux de PC3 (backup Proton) qui sont dans **/home/pjexosql/** sur PC3. Plusieurs ont un alias pratique (voir §10).

| Script | Machine | Catégorie | Rôle en une ligne |
|---|---|---|---|
| zeta_run.sh | PC1 | Calcul | Lance un run complet : turbo ON, calcul, turbo OFF |
| zeta_distribute.py | PC1+PC2 | Calcul | Distribue le calcul v15 sur PC1 (local) + PC2 (SSH) |
| zeta_distribute_run.sh | PC1 | Calcul | Lance la distribution + dashboard (alias zeta-distribute) |
| zeta_run_progress.py | PC1 | Calcul | Dashboard live d'un run distribué (curses) |
| zeta_turbo_on.sh | PC1 | Calcul | Passe le CPU en performance avant un run (sudo) |
| zeta_turbo_off.sh | PC1 | Calcul | Restaure le CPU en powersave après un run (sudo) |
| zeta_monitor.py | PC1 | Monitoring | Dashboard 5/5 : CPU/RAM/charge de chaque machine |
| zeta_tmux.sh | PC1 | Monitoring | Session tmux (5 panneaux + fenêtre PC5) vers le cluster |
| zeta_temp_monitor.py | PC1 → PC1-5 | Monitoring | Courbes de température CPU des 5 machines |
| zeta_sync_pc2.sh | PC1 → PC2 | Cluster | Synchronise le code vers PC2 + recompile les .so |
| zeta_inventaire.sh | PC1 → PC1-5 | Cluster | Inventaire du cluster, lecture seule (alias zeta-inventaire) |
| zeta_rapport.py | PC1 | Cluster | Rapports Markdown à partir des inventaires |
| zeta_diag_publier.sh | PC1 | Cluster | Publie les diagnostics sans IP/MAC + copie sur Proton |
| wg_auto.sh | PC1 | Réseau/VPN | Monte/démonte wg0 selon maison/déplacement |
| zeta_update.sh | PC1 → tous | Maintenance | Met à jour les machines selon l'OS (alias zeta-update) |
| audit_systeme_hprzeta_avant_reinstall.sh | PC1 | Maintenance | Audit complet du système avant réinstallation |
| requirements_zeta.sh | PC1 | Maintenance | Sauvegarde la liste des paquets Python (global+env+apt) |
| zeta_backup_toshiba.sh | PC1 | Backup | Clone incrémental de PC1 vers le disque Toshiba (alias zeta-clone) |
| zeta_backup_horsgit.sh | PC1 | Backup | Sauvegarde sur Proton de ce qui est hors git (alias zeta-backup-horsgit) |
| zeta_backup_status.py | PC1 | Backup | Vérifie l'activité du pipeline rsync PC1→PC3→Proton |
| zeta_proton_status.sh | PC1 | Backup | Lit depuis PC1 l'état du backup Proton de PC3 (alias zeta-proton) |
| check_proton.sh | PC3 | Backup | (cron 8h) teste Proton, publie STATUS.md, lève le flag si KO |
| fix_proton.sh | PC3 | Backup | Régénère le token Proton (demande le mot de passe) |
| zeta_rag.sh | PC1 | RAG/IA | Menu du SSD vault_rag et du RAG (alias zeta-rag) |
| rag_ingest_corpus.py | PC1 | RAG/IA | Ingère le corpus Riemann_Lab dans ChromaDB (v2) |
| rag_query.py | PC1 | RAG/IA | Interroge le RAG : retrieval + génération mathstral |

| Script | Machine | Catégorie | Rôle en une ligne |
|---|---|---|---|
| rag_monitor.py | PC1 | RAG/IA | Surveille le vault RAG (/mnt/vault_rag) |
| build_pdf_riemann.sh | PC1 | Docs/PDF | Convertit un .md du wiki en PDF scientifique (xelatex) |
| build_pdf_riemann_CM.sh | PC1 | Docs/PDF | Variante PDF en police Computer Modern (lualatex) |
| build_pdf_guide_scripts.py | PC1 | Docs/PDF | Génère le PDF de ce guide à partir du .md (ReportLab) |
| pdf_to_md_rapide.sh | PC1 | Docs/PDF | PDF vers Markdown rapide (pymupdf4llm), à utiliser par défaut |
| pdf_to_md.sh | PC1 | Docs/PDF | PDF vers Markdown haute qualité (marker), lent |
| pdf_to_md_voie_b5.sh | PC1 | Docs/PDF | Convertit les PDF de Voie_b_5 vers md_to_claude/ (pdftotext) |
| push_phase0_complete.sh | PC1 | Git | Pousse les MàJ Phase 0 sur GitHub |
| sync_prompts_to_repo.sh | PC1 | Git | Copie les prompts de ~/Téléchargements vers le repo + commit |
| export_riemann_lab_context.sh | PC1 | Utilitaire | Exporte repo+wiki en archive pour reconfiguration |
| gen_arbre.sh | PC1 | Utilitaire | Génère un arbre de dossier interactif |
| capture.sh | PC1 | Utilitaire | Enregistre une session Claude Code en .log + .md |
| archiver_fichiers_obsoletes_v2.sh | PC1 | Utilitaire | Déplace les fichiers obsolètes vers docs/archive/ |
| backup_cluster_map.sh | PC1 | Utilitaire | Génère les cartes SVG du cluster (pipeline/topo) |
| diagnostic_gamma_c_vs_mpmath.sh | PC1 | Utilitaire | Diagnostic Phase C : biais de gamma_c face à mpmath |
| make_assistant_archive_summary.sh | PC1 | Utilitaire | Résume des archives zip/tar en un fichier texte |
| zeta_turbo_status.sh | PC1 | Calcul | État réel du turbo, lecture seule (TURBO_STATUS=...) |
| zeta_session_turbo.sh | PC1 | Calcul | Turbo par SESSION : start / end / status (hook SessionStart et « bye bye ») |
| zeta_script.py | PC1 | Utilitaire | Menu numéroté des alias zeta-* (alias zeta-script) |
| zeta_point_maj.sh | PC1 | Git | Point de situation + bloc daté au guide + commit local (sans push) |
| zeta_export_claudeai.sh | PC1 | IA | Exporte l'état du projet masqué pour claude.ai (alias zeta-export-claudeai) |
| zeta_jsonl_to_md.py | PC1 | Utilitaire | Session Claude Code (.jsonl) vers Markdown masqué (alias zeta-convert-jsontomd) |
| zeta_log_metrics.py, zeta_plot_live.py, zeta_plot_metrics.py, zeta_gue_hist.py | PC1 | Monitoring | Wrappers de scripts/ vers monitoring/scripts/ (voir §8b) |
| Monitoring v2 : install.sh, zeta_monitoring_alias.sh, zeta_mon_run.py, zeta_run_watch.py, zeta_run_report.py, zeta_mon_common.py | PC1 | Monitoring | Lanceur, détecteur, rapports de run (voir §8c) |
| V17 : consolider_v17.py, rejeu_t1b.py, zsign_paires.py, test_consolider_v17.py, test_compute_zeros_v17.py, test_zeta_turbo.py | PC1 | V17 / tests | Consolidation, rejeu, tests (voir §8d) |

| Script | Machine | Catégorie | Rôle en une ligne |
|---|---|---|---|
| Code de calcul et utilitaires de `src/`, `cours/`, `pdf/clone/`, archives | PC1 | Calcul / divers | Inventaire par dossier (voir §8e) |

Autres fichiers de scripts/ : `inbox_ia_liens.md` (liste de liens, ce n'est pas un script) et le dossier `ia_prompts/` (deux fichiers de prompts).

### Prérequis communs (valables pour presque tous les scripts qui touchent au cluster)

- **Fichier d'adresses local** : `~/.config/zeta/cluster_hosts.env` (hors git, droits 600). Il définit ZETA_PC2, ZETA_PC3, ZETA_PC4, ZETA_PC5 et ZETA_BASTION. Les 6 scripts concernés sont : `zeta_monitor.py`, `zeta_tmux.sh`, `wg_auto.sh`, `zeta_distribute.py`, `zeta_backup_status.py`, `zeta_run_progress.py`. Sans ce fichier ils s'arrêtent avec un message (et `wg_auto.sh` ne touche alors pas au tunnel).
- **SSH par clé seule** sur PC2, PC3 et PC5 (le mot de passe est refusé). Les clés utilisées sont ~/.ssh/zeta_cluster (zeta_monitor, zeta_tmux) et ~/.ssh/id_acer (zeta_run_progress, zeta_backup_status, compte pjexosql sur PC3). Les alias SSH `zeta-calc-second`, `zeta-backup`, `zeta-secure` et `zeta-monitor` sont définis dans ~/.ssh/config.
- **PC4** (OpenBSD) : l'élévation de droits passe par doas, qui demande un mot de passe (ssh -t).
- **Environnement Python** : `source ~/projet_zeta/zeta_env/bin/activate` pour les scripts .py (alias `zeta`).
- **Sur PC3**, appeler rclone par son chemin complet **~/bin/rclone** (v1.73.5). Le /usr/bin/rclone (v1.74.3) plante en 32 bits d'après hprzeta (**à vérifier** : la commande version s'y exécute sans erreur, le plantage doit concerner les opérations réelles).

### Éléments système associés (hors scripts/)

| Élément | Où | Rôle |
|---|---|---|
| `99-wg-auto` | /etc/NetworkManager/dispatcher.d/ | À chaque connexion réseau (up / vpn-up), lance `wg_auto.sh --quiet` sous le compte riemann |
| cron PC1 01h50 | crontab de riemann (PC1) | rsync de logs/, wiki/ et pdf/ vers PC3 : **ne tourne que les nuits où PC1 est allumé** |
| cron PC3 02h00 | crontab de pjexosql (PC3) | rclone copy de ~/backup/ vers Proton (protondrive:hprzeta/Riemann_Lab/backup/) |
| cron PC3 08h00 | crontab de pjexosql (PC3) | lance check_proton.sh |
| cron PC1 02h30 | crontab de riemann (PC1) | lance `zeta_backup_horsgit.sh --auto` (sans les secrets) |
| `~/.ssh/config` | PC1 | alias zeta-calc-second, zeta-backup, zeta-secure, zeta-monitor |

# 2. Calcul des zéros de ζ

## `zeta_run.sh`

Machine : PC1 (zeta-lab) · Catégorie : Calcul · MAJ 10/06/2026 (en-tête)

**À quoi ça sert :** Le workflow le plus simple pour un calcul : il enchaîne turbo_on, le calcul, puis turbo_off automatiquement.

**Comment le lancer :**

```
zeta_run.sh 10000
zeta_run.sh 100000
zeta-run 10000
```

■ **À savoir :** Alias : zeta-run. Le log va dans ~/projet_zeta/logs/. Un fichier .pid permet de suivre/arrêter le run. Le script s'arrête (set -e) si T_MAX n'est pas donné.

**Mise à jour du 17/10/2026 :** zeta_run.sh n'appelle plus turbo_off pour le CPU (le turbo reste actif jusqu'au « bye bye ») : il arrête les services pendant le run (--services) puis les redémarre. Il lance **compute_zeros_v16.py** (pas la v17) enveloppé par le lanceur de monitoring si monitoring/scripts/zeta_mon_run.py existe : `ZETA_MON_TYPE=test scripts/zeta_run.sh 100000` (fenêtres de graphiques), défaut `prod` (sans fenêtres). Voir §8b et §8c.

---

## zeta_distribute.py + zeta_distribute_run.sh

Machine : PC1 + PC2 · Catégorie : Calcul · Version : portée de v13 à v15 le 02/08/2026

**À quoi ça sert :** Répartit le calcul sur DEUX machines : PC1 fait [14, T_pivot], PC2 fait [T_pivot, T_MAX] via SSH. Un troisième calcul local, très court, comble le trou de couverture autour du pivot (fenêtre T_pivot ± recouvrement). Le .sh lance en plus un dashboard tmux.

**Prérequis :** fichier d'adresses local (ZETA_BASTION sert au rebond ProxyJump quand PC1 est en déplacement) ; SSH par clé seule vers PC2 (alias zeta-calc-second) ; sudoers sans mot de passe pour le turbo.

**Ce qu'il modifie :** active le turbo sur PC1 (sudo) puis le rétablit ; écrit des fichiers de résultats et de log (log : ~/projet_zeta/logs/distribute_T*_nohup_*.log). Dossier exact des résultats : **à vérifier**.

### Comment le lancer :

```
zeta-distribute 500000
zeta-distribute 500000 --no-dashboard
python scripts/zeta_distribute.py 500000 --v-pc2 1820
```

■ **À savoir :** Alias : zeta-distribute (avec dashboard), zeta-progress-tmux (rattacher le dashboard), zeta-log (suivre le log), zeta-pid (run actif ou non). Dashboard : tmux attach -t zeta-progress. Le pivot est appliqué APRÈS le partitionnement des workers (option B du 24/06/2026) : ne pas le déplacer à la main.

---

## zeta_run_progress.py

Machine : PC1 · Catégorie : Calcul

**À quoi ça sert :** Tableau de bord temps réel d'un run distribué : progression PC1/PC2, zéros trouvés, CPU/RAM/disque par machine.

**Prérequis :** fichier d'adresses local (ZETA_PC2) ; SSH par clé seule vers PC2 (clé ~/.ssh/id_acer).

**Ce qu'il modifie :** rien (lecture seule).

### Comment le lancer :

```
zeta-progress
python scripts/zeta_run_progress.py
```

■ **À savoir :** Alias : zeta-progress (active le venv puis lance). Touches : q = quitter, d = détail par worker. Lancé automatiquement par zeta-distribute.

---

## zeta_turbo_on.sh / zeta_turbo_off.sh

Machine : PC1 (sudo) · Catégorie : Calcul

**À quoi ça sert :** turbo_on met le CPU en 'performance' (+15 à 30 % de vitesse) et réduit le swap avant un run ; turbo_off restaure l'état d'origine après.

### Comment le lancer :

```
zeta-turbo-on
zeta-turbo-off
sudo scripts/zeta_turbo_on.sh
```

■ **À savoir :** Alias : zeta-turbo-on et zeta-turbo-off (ils ajoutent sudo). zeta_run.sh et zeta-distribute les appellent tout seuls : usage manuel rarement nécessaire.

**Mise à jour du 17/10/2026 :** le turbo est désormais activé UNE fois par SESSION (hook SessionStart → zeta_session_turbo.sh start) et restauré au « bye bye » (zeta_session_turbo.sh end) ; zeta_turbo_on.sh est idempotent et conserve l'état initial réel (gouverneur et EPP de chaque CPU, swappiness). Le « +15 à 30 % » ci-dessus n'est PAS démontré : les benchmarks du 16/10/2026 (T=5000 et T=20000, 3 mesures par mode) donnent un écart d'environ 2 %, inférieur à la dispersion. Voir `zeta_turbo_status.sh` et `zeta_session_turbo.sh` (§8b).

# 3. Monitoring & cluster

## zeta_monitor.py

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Dashboard curses : interroge les 5 machines en parallèle par SSH et affiche CPU / RAM / charge de chacune, en continu.

**Prérequis :** fichier d'adresses local ; SSH par clé seule (clé ~/.ssh/zeta_cluster) vers PC2, PC3, PC4 et PC5 ; un rebond par le bastion est utilisé si le tunnel WireGuard répond.

**Ce qu'il modifie :** rien (lecture seule).

**Comment le lancer :**

```
zeta-monitor
python scripts/zeta_monitor.py
```

■ **À savoir :** Alias : zeta-monitor. Rafraîchit toutes les 10 s. PC4 (OpenBSD) et PC3 (vieux Python) ont un parsing adapté.

---

## zeta_tmux.sh

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Ouvre la session tmux « zeta-cluster » : fenêtre 0 avec 5 panneaux (connexion à PC2, PC3, PC4, un shell local dans le venv, et zeta_monitor.py) et une fenêtre « PC5 » (htop en haut, shell libre en bas). Il commence par lancer wg_auto.sh pour la bascule VPN maison/déplacement.

**Prérequis :** fichier d'adresses local ; SSH par clé seule (clé ~/.ssh/zeta_cluster) ; tmux.

**Ce qu'il modifie :** supprime puis recrée la session tmux « zeta-cluster » ; peut monter/couper le tunnel WireGuard via wg_auto.sh.

**Comment le lancer :**

```
zeta-cluster
bash scripts/zeta_tmux.sh
```

■ **À savoir :** Alias : zeta-cluster. Détecte automatiquement maison (LAN direct) vs déplacement (ProxyJump via le bastion).

---

## zeta_temp_monitor.py

Machine : PC1 → PC1 à PC5 · Catégorie : Monitoring · Version : correctif du 03/10/2026

**À quoi ça sert :** Affiche en direct, dans une fenêtre graphique (matplotlib, TkAgg), les courbes de température CPU des 5 machines. PC1 est lu localement chaque seconde ; les autres par SSH toutes les 5 secondes (ConnectTimeout 3 s pour qu'une machine éteinte ne gèle pas les autres). Seuils indicatifs : 75 °C (avertissement) et 90 °C (critique).

**Prérequis :** session graphique ; venv zeta_env (psutil, matplotlib) ; `sensors` installé sur les machines Linux ; alias SSH de ~/.ssh/config (zeta-calc-second, zeta-backup, zeta-secure, zeta-monitor). Lecture PC4 (OpenBSD) : format cpuN.tempN=… degC, commande exacte **à vérifier**.

**Ce qu'il modifie :** rien (lecture seule).

**Comment le lancer :**

```
zeta-temp
python scripts/zeta_temp_monitor.py
```

■ **À savoir :** Alias : zeta-temp. Correction du 03/10/2026 : la lecture ne garde que la première valeur de chaque ligne (la mesure) et ignore les seuils high/crit entre parenthèses, qui faussaient la moyenne.

---

## zeta_sync_pc2.sh

Machine : PC1 → PC2 · Catégorie : Cluster

**À quoi ça sert :** Synchronise le code Python et C vers PC2 puis RECOMPILE les .so sur PC2 : scan_arb.so, illinois_mpfr.so et illinois_arb.so (jamais copier un .so : architectures différentes).

**Prérequis :** SSH par clé seule vers PC2 (alias zeta-calc-second).

**Ce qu'il modifie :** écrit sur PC2 (sources + .so recompilés).

**Comment le lancer :**

```
bash scripts/zeta_sync_pc2.sh
```

■ **À savoir :** PC2 n'est pas un dépôt git : la synchro se fait par rsync/scp. illinois_arb.so lie libflint-arb système (pas python-flint).

---

## zeta_inventaire.sh

Machine : PC1 → PC1 à PC5 · Catégorie : Cluster · Version : v2 (03/10/2026)

**À quoi ça sert :** Dresse l'inventaire de chaque machine (matériel, système, paquets, disques, etc.). Il envoie un petit programme POSIX par SSH (alias de ~/.ssh/config) qui ne modifie rien et fonctionne sous Linux et OpenBSD. Les secrets (psk, passphrase, clés, jetons) sont masqués.

**Prérequis :** SSH par clé seule vers les 4 autres machines ; pour les parties qui demandent root (RAM, SMART, pare-feu), le « complément root » vous fait taper vous-même le mot de passe sudo/doas (jamais stocké).

**Ce qu'il modifie :** rien sur les machines. Il écrit sur PC1, dans ~/zeta_inventaire/ (dossier privé, droits 700, hors git) : un fichier inventaire_PCn_<alias>_<date-heure>.txt par machine.

**Comment le lancer :**

```
zeta-inventaire
./scripts/zeta_inventaire.sh tous
./scripts/zeta_inventaire.sh 5
./scripts/zeta_inventaire.sh root 5
./scripts/zeta_inventaire.sh ajout
```

■ **À savoir :** Alias : zeta-inventaire. Sans argument : menu. « tous » = les 5 PC, un numéro = une machine, « root N » = complément root, « ajout » = ajout d'un PC en mode à blanc. Le dossier peut être changé avec ZETA_INVENTAIRE_DIR.

---

## zeta_rapport.py

Machine : PC1 · Catégorie : Cluster · Version : 03/10/2026

**À quoi ça sert :** Transforme les inventaires en rapports Markdown : un rapport complet par PC (PCn_<nom>.md), une annexe de paquets par PC et un comparatif du cluster (CLUSTER_comparatif.md) avec l'adressage observé.

**Prérequis :** avoir lancé zeta_inventaire.sh (il lit le plus récent inventaire_PCn_*.txt de chaque PC).

**Ce qu'il modifie :** lecture seule, ne se connecte à aucun PC. Il écrit uniquement dans ~/zeta_inventaire/RAPPORT/ (dossier privé).

**Comment le lancer :**

```
./scripts/zeta_rapport.py
./scripts/zeta_rapport.py 3
./scripts/zeta_rapport.py tous
./scripts/zeta_rapport.py comparatif
```

■ **À savoir :** Pas d'alias. Sans argument : menu. Les rapports portent une bannière « DOCUMENT PRIVÉ » (IP, MAC, empreintes SSH) : ne pas les publier.

---

## zeta_diag_publier.sh

Machine : PC1 · Catégorie : Cluster · Version : 03/10/2026

**À quoi ça sert :** Copie les diagnostics privés (~/zeta_inventaire/DIAGNOSTIC) vers la version publiable (~/projet_zeta/docs/diagnostics) en masquant les adresses MAC, IPv6 et IPv4, vérifie qu'il n'en reste aucune, puis sauvegarde DIAGNOSTIC et SCHEMAS sur Proton.

**Prérequis :** rclone avec le remote protondrive (la sauvegarde est sautée avec un message si rclone ou la session Proton manque).

**Ce qu'il modifie :** écrit (et écrase) les .md de docs/diagnostics/ ; copie vers protondrive:hprzeta/Riemann_Lab/zeta_inventaire/. Il ne fait aucun git add ni commit.

**Comment le lancer :**

```
./scripts/zeta_diag_publier.sh
```

■ **À savoir :** Pas d'alias. docs/diagnostics/ est exclu de git en local (info/exclude). Le contrôle résiduel affiche « OK : aucune IP ni MAC détectée » quand tout est masqué.

# 4. Réseau & VPN

### wg_auto.sh

Machine : PC1 · Catégorie : Réseau/VPN · Version : v2 (05/09/2026)

**À quoi ça sert :** Monte ou coupe le tunnel WireGuard wg0 selon l'endroit : à la maison PC4 est joignable en direct (tunnel coupé), en déplacement le tunnel est monté.

**Prérequis :** fichier d'adresses local (ZETA_PC4 et ZETA_BASTION). S'il est absent, le script écrit une ligne dans logs/wg_auto.log et sort avec le code 1 **sans toucher au tunnel**. sudo sans mot de passe pour wg et wg-quick (le script appelle sudo wg show / sudo wg-quick down|up) ; notify-send pour les notifications.

**Ce qu'il modifie :** monte/coupe l'interface wg0 ; écrit logs/wg_auto.log ; envoie une notification bureau.

**Comment le lancer :**

```
wg-auto
bash scripts/wg_auto.sh
bash scripts/wg_auto.sh --quiet
```

■ **À savoir :** Alias : wg-auto. Il est aussi lancé automatiquement à chaque connexion réseau par le dispatcher NetworkManager 99-wg-auto (sous le compte riemann, avec --quiet). v2 : garde une IPv6 globale avant d'agir : sans IPv6 (VPN tiers actif, opérateur) l'endpoint <DOMAINE-DDNS> est injoignable ; le script note « cycle ignoré » dans le log et n'agit pas. En cas de panne d'accès distant, voir le guide WireGuard du wiki. Jamais exécuté avec le nouveau fichier d'adresses en conditions réelles : **à vérifier** au prochain changement de réseau.

# 5. Mise à jour & maintenance

### zeta_update.sh

Machine : PC1 → tous · Catégorie : Maintenance · Version : v2 (19/09/2026)

**À quoi ça sert :** Met à jour les machines selon leur OS (apt pour PC1/2/3/5, syspatch + pkg_add -u pour PC4). Mode CHECK par défaut (n'installe rien). Menu interactif si lancé sans argument.

**Prérequis :** SSH par clé seule vers PC2, PC3, PC5 (alias de ~/.ssh/config) ; pour PC4, ssh -t (doas demande le mot de passe).

**Ce qu'il modifie :** en mode --check, rien. En --apply : installe des paquets sur la machine visée, après confirmation. Log dans /mnt/data/logs/zeta_update_<date>.log.

**Comment le lancer :**

```
zeta-update
zeta-update --only "PC2"
zeta-update --apply --only "PC2"
zeta-update --apply --full --only "PC1"
zeta-update --no-calc-guard --apply --only "PC2"
```

■ **À savoir :** Alias : zeta-update. Ne redémarre JAMAIS seul (signale si reboot requis). Refuse de mettre à jour une machine où un calcul compute_zeros tourne (sauf --no-calc-guard). PC3 (Ubuntu 16.04, fin de vie) : check seulement, apply refusé sauf --force-eol (à éviter). PC4 (bastion) : jamais sysupgrade, traité en dernier. --full = apt dist-upgrade. PC5 figure dans la liste des machines du code, mais le menu n'offre que PC1 à PC4 et « TOUTES » : **à vérifier** que --only "PC5" fonctionne.

---

### audit_systeme_hprzeta_avant_reinstall.sh

Machine : PC1 · Catégorie : Maintenance · Fichier du 05/07/2026

**À quoi ça sert :** Inventorie tout le système (paquets, matériel, partitions, GPU, services, dépôts git, configs) dans une archive .tar.gz, utile avant une réinstallation.

**Comment le lancer :**

```
./scripts/audit_systeme_hprzeta_avant_reinstall.sh
```

■ **À savoir :** Sortie dans ~/hprzeta-system-audit/. Variables OUT_BASE=, EXTRA_SCAN_DIRS= personnalisables.

---

### `requirements_zeta.sh`

Machine : PC1 · Catégorie : Maintenance · Fichier du 05/07/2026

**À quoi ça sert :** Sauvegarde la liste de tous les paquets Python (global, venv zeta_env, et apt système) dans des fichiers requirements_zeta_*.txt.

**Comment le lancer :**

```
bash scripts/requirements_zeta.sh
```

# 6. Sauvegarde

## Calendrier automatique du pipeline

| Heure | Machine | Action | Condition |
|---|---|---|---|
| 01h50 | PC1 | rsync de logs/, wiki/, pdf/ vers PC3 | seulement si PC1 est allumé |
| 02h00 | PC3 | rclone copy de ~/backup/ vers Proton | toujours (cron de PC3) |
| 02h30 | PC1 | zeta_backup_horsgit.sh --auto (sans secrets) | seulement si PC1 est allumé |
| 08h00 | PC3 | check_proton.sh : test Proton + STATUS.md + flag | toujours |

---

### `zeta_backup_status.py`

Machine : PC1 · Catégorie : Backup

**À quoi ça sert :** Vérifie que le pipeline de backup tourne : cron rsync PC1→PC3 (01h50) puis rclone PC3→Proton (02h00). Montre les dates réelles.

**Prérequis :** fichier d'adresses local (ZETA_PC3) ; SSH par clé seule vers PC3 en tant que pjexosql (clé ~/.ssh/id_acer).

**Ce qu'il modifie :** rien (lecture seule).

**Comment le lancer :**

```
zeta-backup-status
python scripts/zeta_backup_status.py
python scripts/zeta_backup_status.py --date 2026-08-04
```

■ **À savoir :** Alias : zeta-backup-status. Se connecte à PC3 en tant que pjexosql (clé id_acer), pas hprzeta : c'est pjexosql qui possède ~/backup/. Le rsync de 01h50 ne tourne que les nuits où PC1 est allumé. La colonne « Atterrissage (PC3) » est une preuve indirecte (fichier modifié ce jour-là sur PC3) : « aucun fichier » veut dire « rien de nouveau à copier », pas forcément une panne.

---

### `zeta_proton_status.sh`

Machine : PC1 · Catégorie : Backup · MAJ 19/09/2026

**À quoi ça sert :** Dit en une ligne si le backup Proton de PC3 va bien. Il se connecte à PC3 (alias zeta-backup) et lit uniquement le fichier d'alerte PROTON_KO.flag posé par check_proton.

**Prérequis :** SSH par clé seule vers PC3.

**Ce qu'il modifie :** rien (lecture seule).

**Comment le lancer :**

```
zeta-proton
zeta-proton --quiet
```

■ **À savoir :** Alias : zeta-proton. Codes : 0 = OK, 1 = Proton KO (lancer fix_proton sur PC3), 2 = PC3 injoignable (état inconnu). Il ne teste PAS Proton lui-même : si le cron de 08h n'a pas tourné, le flag

peut être périmé.

---

## `check_proton.sh`

Machine : PC3 (compte pjexosql, cron 8h) · Catégorie : Backup · Version : v2

**À quoi ça sert :** Teste chaque jour que Proton répond, génère un STATUS.md (poussé sur le Drive dans Riemann_Lab/monitoring/), et lève un flag (PROTON_KO.flag) si le token a expiré ; il efface le flag quand tout va bien.

**Prérequis :** rclone par son chemin complet /home/pjexosql/bin/rclone (v1.73.5).

**Ce qu'il modifie :** écrit STATUS.md et le flag dans le home de pjexosql, et pousse STATUS.md sur Proton.

**Comment le lancer :**

```
# tourne tout seul via cron (8h) ; test manuel depuis PC1 :
ssh -i ~/.ssh/id_acer pjexosql@<PC3> 'bash ~/check_proton.sh'
```

■ **À savoir :** Ne plus utiliser « sudo -u pjexosql » : le NOPASSWD a été retiré sur PC3. Si le token est mort, le push du STATUS.md échoue aussi : l'horodatage du STATUS.md sert alors d'indice : vieux de plusieurs jours, il signale un flux cassé.

---

## `fix_proton.sh`

Machine : PC3 (compte pjexosql) · Catégorie : Backup

**À quoi ça sert :** Répare le backup Proton quand le token a expiré (erreur 401) : demande votre mot de passe Proton (saisi masqué) et régénère les tokens rclone, puis vérifie que Proton répond. À lancer quand zeta-proton dit KO.

**Prérequis :** rclone par son chemin complet /home/pjexosql/bin/rclone ; le mot de passe Proton.

**Ce qu'il modifie :** la configuration rclone de pjexosql (tokens).

**Comment le lancer :**

```
# SUR PC3, en tant que pjexosql :
bash ~/fix_proton.sh
```

■ **À savoir :** Le mot de passe est saisi masqué et ne quitte jamais PC3. Le vrai backup Proton = protondrive:hprzeta/Riemann_Lab/backup/.

---

## `zeta_backup_toshiba.sh`

Machine : PC1 (sudo) · Catégorie : Backup · Version : 2.2 (en-tête daté du 26/09/2026)

**À quoi ça sert :** Clone incrémental de PC1 (root + home + data) vers un disque externe Toshiba, identifié par label (jamais par sdX). Il fait d'abord un dry-run automatique (root, home, data) avec le résumé des suppressions, puis demande confirmation avant le vrai clonage.

**Prérequis :** Toshiba branché, avec les labels Toshiba-PC1-root, Toshiba-PC1-home, Toshiba-PC1-data (partitions montées automatiquement par label) ; sudo.

**Ce qu'il modifie :** écrit sur le Toshiba (rsync -aAXHx avec --delete, suppressions incluses) et corrige le fstab (UUID) sur le clone. Exclusions permanentes : le travail SAP (Documents/SAP*, côté home), lost+found et swapfile. Log : ~/zeta_clone_<date>.log ; résumés du dry-run dans /tmp/zeta_dryrun_<date>.

**Comment le lancer :**

```
zeta-clone
sudo bash scripts/zeta_backup_toshiba.sh
```

■ **À savoir :** Alias : zeta-clone (ajoute sudo). v2.1 : exclusion permanente du travail SAP. v2.2 : dry-run automatique avant confirmation (le clone peut contenir des fichiers plus récents que PC1). Les dates de modification du fichier (26/09/2026) ne correspondent pas à la version « 03/10 » annoncée : **à vérifier**.

---

### zeta_backup_horsgit.sh

Machine : PC1 · Catégorie : Backup · Version : 04/10/2026

**À quoi ça sert :** Sauvegarde sur Proton Drive (protondrive:hprzeta/Riemann_Lab/hors_git/) tout ce qui n'est pas dans git. Menu : 1 md/ ; 2 mémoire Claude ; 3 suivi (riemann_handoff, sans secrets) ; 4 fichiers non suivis par git ; 5 calculs légers (fichiers de moins de 200 Ko) ; 6 secrets chiffrés (archive gpg symétrique, phrase de passe saisie par vous : secrets_local + les secrets du projet ignorés par git) ; 7 configuration locale (~/.config/zeta) ; 8 fichiers ignorés par git, hors secrets (pdf/script, docs/diagnostics, *.bak-*...) ; « a » = tout sauf les secrets (1 à 5, 7 et 8) ; « d » bascule entre envoi réel et simulation.

**Prérequis :** remote rclone protondrive actif (quota Proton de 2 Gio) ; gpg pour le jeu 6.

**Ce qu'il modifie :** écrit sur Proton (rclone copy, aucune suppression), SAUF la rotation du jeu 6 : après un envoi réussi, elle supprime les archives secrets_AAAAMMJJ.tar.gz.gpg les plus anciennes de secrets_chiffres/ et n'en garde que 2 (réglage KEEP_SECRETS). Exclus volontairement : secrets en clair (ils ne partent que chiffrés, jeu 6), .mcp.json, .env, ~/.ssh, rclone.conf, calculs/ (4 Go) et zeta_env.

**Comment le lancer :**

```
zeta-backup-horsgit
./scripts/zeta_backup_horsgit.sh --dry
./scripts/zeta_backup_horsgit.sh --auto
```

■ **À savoir :** Alias : zeta-backup-horsgit. Le menu démarre en ENVOI RÉEL : tapez d pour passer en simulation. --auto (sans menu, sans secrets : jeux 1 à 5, 7 et 8) est ce que lance le cron de 02h30, qui ne part que si PC1 est allumé. Pour envoyer les secrets : lancer zeta-backup-horsgit puis taper 6 (SANS d : le menu est déjà en envoi réel). Le jeu 6 et la rotation n'ont jamais été exécutés en réel. Explication complète : md/secur/Sauvegarde_secrets_chiffres_methode.md (PDF dans pdf/secur/).

# 7. RAG / IA — Objectif 2

### zeta_rag.sh

Machine : PC1 (venv zeta_env) · Catégorie : RAG/IA · Version : 1.0 (03/10/2026)

**À quoi ça sert :** Menu du BrainVault : gère le SSD vault_rag et le RAG. Le SSD est reconnu par UUID et étiquette, jamais par /dev/sdX. Options : 1 État (monté ? bon disque ? espace, collections, SMART) ; 2 Monter sur /mnt/vault_rag ; 3 Démonter proprement (sync puis démontage, avant de débrancher) ; 4 Poser une question (rag_query.py) ; 5 Surveillance (photo, continu toutes les 10 s, latence, ou session tmux via monitor_rag.sh) ; 6 Ingestion (copie de sécurité, remise à zéro, ré-ingestion) ; 7 Sauvegarde du vault ; 0 Quitter.

**Prérequis :** le SSD vault_rag branché ; sudo (seulement pour monter, démonter et lire le SMART) : ne pas lancer le script avec sudo ; rsync ; smartmontools pour le SMART.

**Ce qu'il modifie :** monte/démonte le SSD ; aucune écriture si le SSD n'est pas monté ET reconnu. Option 6 : copie de sécurité dans /mnt/data/exports/vault_backup_<date-heure>, puis rag_ingest_corpus.py --reset (supprime la collection riemann_lab_corpus, 838 chunks au dernier relevé). Option 7 : copie du vault vers /mnt/data/exports/vault_backup_<date> par rsync SANS --delete, après une simulation. Journal : logs/zeta_rag_menu.log.

**Comment le lancer :**

```
zeta-rag
zeta-rag etat
zeta-rag monter
zeta-rag demonter
zeta-rag question
zeta-rag surveillance
zeta-rag ingestion
zeta-rag sauvegarde
```

■ **À savoir :** Alias : zeta-rag. Sans argument : menu ; avec un mot, une seule option puis sortie. Ctrl+C arrête le programme lancé, pas le menu. Le SMART suit la tendance par rapport à une référence du 03/10/2026 (Micron 1100) : blocs NAND réalloués, erreurs non corrigées, coupures brutales. L'option « session tmux » de la surveillance appelle scripts/monitor_rag.sh, qui **n'existe pas** : le menu affiche « introuvable ». L'ingestion refuse de tourner si rag_ingest_corpus.py n'a pas l'option --reset.

### `rag_ingest_corpus.py`

Machine : PC1 (venv zeta_env) · Catégorie : RAG/IA · Version : 2 (03/10/2026)

**À quoi ça sert :** Ingère le corpus Riemann_Lab dans la base vectorielle ChromaDB du SSD /mnt/vault_rag : pages courantes du wiki (hors Handoff.md), code actuel de src/calculs/optimisation (.py, .c, .h) et prompts archivés (bandeau ARCHIVE uniquement). Sortie : collection « riemann_lab_corpus », cache LlamaIndex et log sur le SSD.

**Prérequis :** venv zeta_env activé ; SSD monté (garde mountpoint, comme rag_monitor.py et rag_query.py).

**Ce qu'il modifie :** écrit dans la collection ChromaDB et dans les dossiers llamaindex_cache et agent_logs du SSD.

**Comment le lancer :**

```
source zeta_env/bin/activate
python scripts/rag_ingest_corpus.py
python scripts/rag_ingest_corpus.py --reset
```

■ **À savoir :** v2 : refuse de tourner si le SSD n'est pas monté (avant, il écrivait en silence sur le disque système). Il refuse aussi d'ingérer dans une collection NON vide sans --reset : relancer ajoutait les chunks une 2e fois (838 devenait environ 1676). --reset supprime la collection puis ré-ingère : à utiliser après une copie de sécurité (zeta-rag, option 6). Respecte la règle d'ingestion (§3.5 de l'état RAG).

---

### `rag_query.py`

Machine : PC1 (venv zeta_env) · Catégorie : RAG/IA · MAJ 04/10/2026 (fichier)

**À quoi ça sert :** Pose une question au RAG : embedding de la question (all-MiniLM-L6-v2, identique à l'ingestion), recherche dans ChromaDB, puis génération d'une réponse avec mathstral via ollama (API locale).

**Prérequis :** venv zeta_env ; `ollama serve` actif avec le modèle mathstral ; SSD monté.

**Ce qu'il modifie :** rien d'important ; l'option --log écrit un journal.

**Comment le lancer :**

```
python scripts/rag_query.py "Quel est le seuil SEUIL_1NEWTON ?"
python scripts/rag_query.py "IP cluster ?" --k 5 --no-llm
python scripts/rag_query.py "..." --modele mathstral --log
```

■ **À savoir :** mathstral (4,1 Go) charge plusieurs couches en RAM : PC1 a 16 Go depuis le 25/07/2026, mais fermer VS Code et Firefox avant un test reste prudent (crash du 06/07/2026) ; le script avertit sans bloquer. --no-llm = retrieval seul. L'exemple de citation du prompt a été modifié le 04/10/2026 : le test anti-hallucination reste à relancer.

---

### `rag_monitor.py`

Machine : PC1 (venv zeta_env) · Catégorie : RAG/IA · Fichier du 19/07/2026

**À quoi ça sert :** Photo (ou surveillance continue) de l'état du vault RAG : montage du SSD, espaces disque (SSD et racine), tailles des dossiers, collections et nombre de chunks, dernières lignes du log d'ingestion, et latence d'une vraie requête avec --test-query.

**Comment le lancer :**

```
python scripts/rag_monitor.py
python scripts/rag_monitor.py --watch 10
python scripts/rag_monitor.py --test-query
```

■ **À savoir :** Vérifie que le SSD est bien monté (leçon du 05/07 : SSD non remonté = écriture silencieuse sur le disque système).

# 8. Documents, PDF & Git

## build_pdf_riemann.sh / _CM.sh

Machine : PC1 · Catégorie : Docs/PDF · Fichiers du 05/07/2026

**À quoi ça sert :** Convertit une page .md du wiki en PDF scientifique (formules, code encadré). La variante _CM utilise la police Computer Modern (lualatex).

**Comment le lancer :**

```
./scripts/build_pdf_riemann.sh source.md [sortie.pdf]
./scripts/build_pdf_riemann_CM.sh source.md
```

■ **À savoir :** Dépend de pandoc + xelatex/lualatex (texlive) + polices DejaVu.

---

## build_pdf_guide_scripts.py

Machine : PC1 (venv zeta_env) · Catégorie : Docs/PDF · Version : 04/10/2026

**À quoi ça sert :** Génère le PDF de CE guide à partir de son .md, avec ReportLab, dans le même aspect que l'édition du 19/09/2026 (A4, titres bleus, blocs de code gris, pied de page « page N »).

**Prérequis :** venv zeta_env (reportlab) ; polices DejaVu pour les accents et les symboles.

**Ce qu'il modifie :** écrit uniquement le PDF de sortie ; ne touche jamais au .md ni à l'ancien PDF.

**Comment le lancer :**

```
python scripts/build_pdf_guide_scripts.py md/script/Guide_Scripts_Cluster_Zeta.md
    pdf/script/Guide_Scripts_Cluster_Zeta_AAAAMMJJ.pdf
```

■ **À savoir :** Le guide est PRIVÉ (il contient des IP) : le PDF de pdf/script/ n'est pas exclu de git, ne jamais le committer ni le copier dans docs/ ou le wiki.

---

## pdf_to_md_rapide.sh / pdf_to_md.sh

Machine : PC1 · Catégorie : Docs/PDF · Fichiers du 05/07/2026

**À quoi ça sert :** Convertit les PDF d'un dossier en Markdown. '_rapide' (pymupdf4llm) = rapide et léger, à utiliser par défaut ; '.sh' (marker) = qualité max mais lent (environ 22 min/PDF, 5 Go de RAM).

**Comment le lancer :**

```
./scripts/pdf_to_md_rapide.sh ~/projet_zeta/pdf/mon_dossier
./scripts/pdf_to_md.sh ~/projet_zeta/pdf/complexe
```

■ **À savoir :** But : un .md pèse 10 à 50 fois moins qu'un PDF donné à Claude (coût en jetons). Les deux scripts écrivent à côté du PDF et ne sont pas récursifs (un seul dossier à la fois) : pour viser md/, passer par une copie temporaire des PDF.

---

## pdf_to_md_voie_b5.sh

Machine : PC1 · Catégorie : Docs/PDF · MAJ 03/06/2026 (en-tête)

**À quoi ça sert :** Convertit une liste précise de PDF du dossier pdf/optimisation/Voie_b_5/ en .md rangés dans md_to_claude/ : extraction du texte avec pdftotext -layout, puis ajout d'un bandeau (fichier, dossier wiki inbox-ia, auteur, date).

**Prérequis :** pdftotext (poppler-utils) ; les PDF de Voie_b_5 aux noms prévus dans la liste du script.

**Ce qu'il modifie :** écrit les .md dans pdf/optimisation/Voie_b_5/md_to_claude/ ; un PDF introuvable est signalé et sauté.

**Comment le lancer :**

```
bash scripts/pdf_to_md_voie_b5.sh
```

■ **À savoir :** Pas d'alias. Les noms de fichiers sont écrits en dur dans le script : il ne sert qu'à ce lot de PDF.

---

## push_phase0_complete.sh / sync_prompts_to_repo.sh

Machine : PC1 · Catégorie : Git · Fichiers du 05/07/2026

**À quoi ça sert :** push_phase0 pousse les MàJ Phase 0 sur GitHub ; sync_prompts copie les prompts de ~/Téléchargements vers src/ia/prompts/ et commite.

**Comment le lancer :**

```
bash scripts/push_phase0_complete.sh
bash scripts/sync_prompts_to_repo.sh
```

■ **À savoir :** Travaillent sur la branche Riemann_Lab_IA. sync_prompts_to_repo.sh COMMITE : à lancer en connaissance de cause.

---

## Utilitaires divers

Machine : PC1 · Catégorie : Utilitaire

---

## gen_arbre.sh, capture.sh, archiver_fichiers_obsoletes_v2.sh

**À quoi ça sert :** gen_arbre.sh génère un arbre de dossier interactif ; capture.sh enregistre une session Claude Code en .log + .md ; archiver_fichiers_obsoletes_v2.sh déplace les fichiers obsolètes vers docs/archive/. Fichiers du 05/07/2026.

**Comment le lancer :**

```
bash scripts/gen_arbre.sh
bash scripts/capture.sh
```

■ **À savoir :** capture.sh : lancer, puis `claude`, puis exit deux fois. archiver_fichiers_obsoletes_v2.sh DÉPLACE des fichiers : relire sa liste avant de le lancer.

---

## export_riemann_lab_context.sh

Machine : PC1 · Catégorie : Utilitaire · Fichier du 03/10/2026

**À quoi ça sert :** Récupère le dépôt hprzeta/Riemann_Lab et son wiki (git clone), en extrait les Markdown, docs, skills, scripts, configs, requirements, inventaires et logs Git, et produit une archive à transmettre pour reconfiguration.

**Prérequis :** git, tar, gzip, zip ; accès réseau à GitHub. Le script contient une proposition d'installer ces outils avec sudo apt install (**à vérifier** : il propose ou il exécute).

**Ce qu'il modifie :** crée un dossier de travail dans BASE_DIR (par défaut ~/hprzeta-import) avec un sous-dossier riemann_lab_context_export_<date-heure>.

**Comment le lancer :**

```
./scripts/export_riemann_lab_context.sh
BASE_DIR=/chemin ./scripts/export_riemann_lab_context.sh
```

■ **À savoir :** L'archive rassemble des inventaires et des logs : à traiter comme privée, ne pas la publier.

---

## backup_cluster_map.sh

Machine : PC1 · Catégorie : Utilitaire · MAJ 26/09/2026 (en-tête)

**À quoi ça sert :** Génère les cartes SVG du cluster : « pipeline » (pipeline de backup) et « topo » (topologie des machines).

**Ce qu'il modifie :** pipeline ou topo écrivent sur la sortie standard (à rediriger) ; « all » génère les 2 fichiers SVG dans docs/images/ (backup_cluster_map.svg et topo_machines_zeta.svg), en écrasant les versions existantes.

**Comment le lancer :**

```
bash scripts/backup_cluster_map.sh pipeline > docs/images/backup_cluster_map.svg
bash scripts/backup_cluster_map.sh all
```

■ **À savoir :** Pas d'alias. Depuis la resynchronisation du 26/09/2026 (en-tête), les cartes n'affichent plus d'IP ni de nom DDNS.

---

## diagnostic_gamma_c_vs_mpmath.sh

Machine : PC1 · Catégorie : Utilitaire (Phase C) · Fichier du 05/07/2026

**À quoi ça sert :** Diagnostic du biais entre gamma_c, calculé par le module C illinois_mpfr.so, et gamma_true, corrigé avec mpmath.siegelz.

**Prérequis :** être dans ~/projet_zeta/src/calculs/optimisation (le script lit c_modules/illinois_mpfr.so) ; mpmath.

**Ce qu'il modifie :** écrit un fichier temporaire (/tmp/diagnostic_gamma_c_vs_mpmath.py) et le rapport ~/phase_c_gamma_c_vs_mpmath.txt.

**Comment le lancer :**

```
cd ~/projet_zeta/src/calculs/optimisation
./diagnostic_gamma_c_vs_mpmath.sh
```

■ **À savoir :** Pas d'alias. Le biais Z_mpfr d'environ 1e-3 n'est pas un bug : c'est la limite de Riemann-Siegel tronquée à C0+C1 ; la solution retenue est le wrapper mpmath.siegelz.

---

### `make_assistant_archive_summary.sh`

Machine : PC1 · Catégorie : Utilitaire · Fichier du 05/07/2026

**À quoi ça sert :** Résume un ou plusieurs fichiers .zip, .tar, .tar.gz, .tgz, .tar.bz2, .tar.xz dans un fichier texte lisible par un assistant, sans décompresser les archives.

**Ce qu'il modifie :** écrit uniquement le fichier texte de sortie (par défaut assistant_pack_audit_resume_global.txt, dans le dossier courant).

**Comment le lancer :**

```
./scripts/make_assistant_archive_summary.sh archive1.zip archive2.tar.gz
./scripts/make_assistant_archive_summary.sh -o resume.txt archive.zip
./scripts/make_assistant_archive_summary.sh --auto
```

■ **À savoir :** Options : -o (sortie), --auto (cherche les archives du dossier courant), --max-lines N (20000 par défaut), --sha256, -h. Les fichiers assistant_pack_audit_*.txt sont ignorés par git.

# 8b. Turbo de session, menu et outils Claude Code (scripts/ complémentaires)

Ajouts du 17/10/2026. Descriptions tirées des en-têtes des scripts ; « Statut » indique la preuve disponible (jamais présumée).

---

### `zeta_turbo_status.sh`

Machine : PC1 · Catégorie : Calcul / turbo

**À quoi ça sert :** Affiche l'état RÉEL du turbo : gouverneur et EPP de chaque CPU, boost matériel (no_turbo), swappiness, profil power-profiles-daemon et état sauvegardé par zeta_turbo_on.sh. Dernière ligne : TURBO_STATUS=ACTIF | INACTIF | DERIVE | INVALIDE | ILLISIBLE.

**Prérequis :** aucun sudo ; l'état sauvegardé /run/zeta_turbo/state est lisible par le groupe adm.

**Ce qu'il modifie / produit :** rien (lecture seule).

**Comment le lancer :**

```
scripts/zeta_turbo_status.sh
```

■ **À savoir :** Codes de sortie : 0 ACTIF · 1 INACTIF · 2 DERIVE · 3 INVALIDE/ILLISIBLE (le code 1 quand INACTIF n'est pas une erreur). DERIVE = un CPU n'est plus en performance (power-profiles-daemon ?) : relancer zeta_turbo_on.sh. Variables de test : ZETA_TURBO_SYSROOT, ZETA_TURBO_STATE_DIR.

**Sudo :** non · **Sensible :** non · **Statut :** testé (tests_turbo : 34 OK le 16/10/2026 ; lecture réelle le 17/10/2026 : TURBO_STATUS=ACTIF).

---

### `zeta_session_turbo.sh`

Machine : PC1 · Catégorie : Calcul / turbo

**À quoi ça sert :** Enveloppe sans privilège du cycle de vie du turbo de SESSION : start (appelé par le hook SessionStart), end (1re étape garantie du « bye bye », restaure l'état initial réel) et status.

**Prérequis :** règles sudo NOPASSWD existantes sur zeta_turbo_on.sh / zeta_turbo_off.sh ; copies durcies /usr/local/sbin/zeta_turbo_on|off utilisées de préférence si elles existent (non encore installées).

**Ce qu'il modifie / produit :** journal logs/turbo_session.log (ignoré par git) ; l'état turbo est géré par zeta_turbo_on.sh / zeta_turbo_off.sh.

**Comment le lancer :**

```
scripts/zeta_session_turbo.sh start
scripts/zeta_session_turbo.sh end
scripts/zeta_session_turbo.sh status
```

■ **À savoir :** start n'échoue jamais (code 0) pour ne pas bloquer l'ouverture de session ; end renvoie le code de zeta_turbo_off.sh (≠ 0 si restauration incomplète). Une seule session Zêta à la fois : le 1er « bye bye » restaure pour tout le système.

**Sudo :** oui (indirect : sudo -n sur les scripts turbo) · **Sensible :** oui (modifie gouverneur CPU et swappiness) · **Statut :** testé (cycle start / status / end réel le 16/10/2026 ; start automatique à l'ouverture de la session du 17/10/2026).

---

## zeta_script.py

Machine : PC1 · Catégorie : Utilitaire (menu)

**À quoi ça sert :** Affiche le menu numéroté des alias zeta-* (lus dans ~/.bashrc) et des fonctions du monitoring (lues dans monitoring/zeta_monitoring_alias.sh), puis lance celui choisi après confirmation.

**Prérequis :** python3 ; ~/.bashrc ; fichier monitoring/zeta_monitoring_alias.sh (optionnel).

**Ce qu'il modifie / produit :** rien par lui-même (lecture seule) ; l'alias choisi, lui, s'exécute dans un shell interactif.

**Comment le lancer :**

```
zeta-script
python3 -I scripts/zeta_script.py
```

■ **À savoir :** ⚠ marque les alias sensibles (sudo, run lourd, écriture disque, fenêtre graphique) ; certaines fonctions du monitoring proposent une saisie d'arguments. Le menu compte 58 entrées au 17/10/2026 (n° 1 à 36 = ~/.bashrc, n° 37 à 51 = monitoring, n° 52 à 58 = couleurs de terminal de ~/zeta_colors.sh).

**Sudo :** non · **Sensible :** non · **Statut :** testé (menu affiché et zeta-mon-status lancé via le menu le 16/10/2026 ; 58 entrées relues le 17/10/2026, après l'ajout des 7 fonctions de couleurs ; lancement de zeta_reset_bg via le menu vérifié).

---

## zeta_point_maj.sh

Machine : PC1 · Catégorie : Utilitaire / Git

**À quoi ça sert :** Point de situation sur l'état système RÉEL (lecture seule), ajout d'un bloc daté au guide canonique (sauvegarde préalable), puis git add + commit LOCAL ; s'arrête avant le push (affiché, jamais exécuté).

**Prérequis :** instance Claude Code sur PC1 ; dépôt git.

**Ce qu'il modifie / produit :** guide canonique (bloc daté + sauvegarde) ; un commit local.

**Comment le lancer :**

```
bash scripts/zeta_point_maj.sh
```

■ **À savoir :** Version 1.0 du 30/08/2026 d'après l'en-tête. Le mot « sudo » n'y apparaît que dans un texte de recommandation (simulation rsync avant clone), le script n'exécute pas de sudo. Le push reste manuel.

**Sudo :** non · **Sensible :** oui (commit local) · **Statut :** non testé dans cette session (version 1.0, 30/08/2026).

---

## zeta_export_claudeai.sh

Machine : PC1 · Catégorie : Utilitaire / IA

**À quoi ça sert :** Produit UN fichier Markdown masqué (état du projet : bloc PROMPT_REPRISE de ~/riemann_handoff/Handoff.md + CLAUDE.md du projet) à déposer à la main dans un Projet claude.ai.

**Prérequis :** bash ; ~/riemann_handoff/Handoff.md ; CLAUDE.md du projet ; masquage de zeta_jsonl_to_md.py.

**Ce qu'il modifie / produit :** écrit ~/zeta_inventaire/export_claudeai/contexte_claudeai_AAAAMMJJ.md (droits 600) ; refuse d'écrire si une 2e passe de masquage change encore le texte.

**Comment le lancer :**

```
zeta-export-claudeai
bash scripts/zeta_export_claudeai.sh
```

■ **À savoir :** Garanties de l'en-tête : sources en lecture seule ; aucun réseau, commit, push ni suppression. Alias : zeta-export-claudeai.

**Sudo :** non · **Sensible :** non (⚠ non marqué dans le menu) · **Statut :** non testé dans cette session.

---

### `zeta_jsonl_to_md.py`

Machine : PC1 · Catégorie : Utilitaire / archives de session

**À quoi ça sert :** Transcription d'une session Claude Code (.jsonl) en Markdown propre : sorties d'outils et raisonnement omis ; adresses réseau, e-mails et secrets masqués.

**Prérequis :** python3 ; fichiers de session .jsonl de Claude Code.

**Ce qu'il modifie / produit :** écrit un .md en mode 600 (dossier -o, sinon le défaut du script) ; le .jsonl n'est jamais modifié.

**Comment le lancer :**

```
zeta-convert-jsontomd [SESSION] [-o DOSSIER] [--liste]
python3 -I scripts/zeta_jsonl_to_md.py --liste
```

■ **À savoir :** SESSION = chemin .jsonl, début d'identifiant ou numéro (voir --liste) ; défaut : la session la plus récente. Auteur hprzeta, créé le 06/10/2026 ; son masquage est réutilisé par zeta_export_claudeai.sh. Alias : zeta-convert-jsontomd.

**Sudo :** non · **Sensible :** non (⚠ non marqué dans le menu) · **Statut :** non testé dans cette session.

---

### `zeta_log_metrics.py, zeta_plot_live.py, zeta_plot_metrics.py, zeta_gue_hist.py (dans scripts/)`

Machine : PC1 · Catégorie : Monitoring (wrappers)

**À quoi ça sert :** depuis le 17/10/2026 (commit bff901f), ces 4 fichiers de scripts/ ne sont plus que des wrappers d'une ligne vers les versions de monitoring/scripts/, qui sont la SEULE source (voir §8c). Les anciennes versions sont conservées en `.bak-20261017`.

**Comment les lancer :** de préférence via les alias (zeta-mon-on, zeta-live, zeta-metrics, zeta-gue) ; en direct : `python scripts/zeta_log_metrics.py --test`.

**Sudo :** non · **Sensible :** non · **Statut :** testé (`zeta_log_metrics.py --test` et `--help` des 3 autres via les wrappers le 17/10/2026).

## 8c. Monitoring v2 (dossier monitoring/)

Ajouté le 17/10/2026. Principe : un lanceur (`zeta_mon_run.py`) enveloppe un run, un logger enregistre les 5 machines toutes les secondes, un superviseur écrit l'état du run toutes les 5 s, et à la fin un rapport MD + PDF (avec une section « Interprétation » rédigée à la main) est généré. Sorties dans monitoring/runs/test|prod/AAAAMMJJ_HHMMSS_nom/. **Limite connue (v16)** : le CSV des zéros n'est écrit qu'à la fin de chaque worker, donc le graphique en direct des zéros se remplit par à-coups et le débit instantané (etat.csv) est inexploitable ; se fier au débit moyen du rapport.

---

### `monitoring/install.sh`

Machine : PC1 · Catégorie : Monitoring (installation)

**À quoi ça sert :** À lancer UNE fois sur PC1 : crée data/, runs/, logs/, copie les anciens CSV du logger vers data/ (sans écraser), range l'ancien run plat dans runs/historique/, ajoute au ~/.bashrc la ligne qui charge zeta_monitoring_alias.sh si elle manque, vérifie numpy/pandas/matplotlib et reportlab/pillow, régénère Monitoring-Runs.md.

**Prérequis :** zeta_env (ou python3) avec numpy, pandas, matplotlib, reportlab, pillow.

**Ce qu'il modifie / produit :** dossiers monitoring/data|runs|logs ; ~/.bashrc (une ligne, idempotent) ; Monitoring-Runs.md.

**Comment le lancer :**

```
bash ~/projet_zeta/monitoring/install.sh
source ~/.bashrc
```

■ **À savoir :** Ouvrir un NOUVEAU terminal (ou source ~/.bashrc) ensuite, sinon les nouvelles commandes sont « introuvables ». Variable ZETA_PY pour choisir l'interpréteur.

**Sudo :** non · **Sensible :** oui (écrit dans ~/.bashrc si la ligne manque) · **Statut :** testé (exécuté le 17/10/2026 ; ligne déjà présente dans ~/.bashrc).

---

## monitoring/zeta_monitoring_alias.sh

Machine : PC1 · Catégorie : Monitoring (commandes)

**À quoi ça sert :** Définit les 15 fonctions bash zeta-mon-on/off/status, zeta-live, zeta-metrics, zeta-gue, zeta-mon-exec/start/stop/fenetres/fermer, zeta-run-hint, zeta-report, zeta-index, zeta-runs (ce sont des fonctions, pas des alias, car elles acceptent des arguments).

**Prérequis :** source depuis ~/.bashrc (ligne 186) ; variables ZETA_MON_ROOT, ZETA_PY, ZETA_CALCULS.

**Ce qu'il modifie / produit :** rien par lui-même (définit des fonctions).

**Comment le lancer :**

```
source ~/projet_zeta/monitoring/zeta_monitoring_alias.sh
```

■ **À savoir :** Voir le détail de chaque commande au §10b. Lu aussi par zeta_script.py pour compléter le menu.

**Sudo :** non · **Sensible :** non · **Statut :** testé (fonctions chargées et exécutées le 17/10/2026).

---

## monitoring/scripts/zeta_mon_run.py

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Lanceur de monitoring d'un run : exec (enveloppe une commande, récupère son code retour, clôt le run), start (suivi d'un run lancé à côté, --pid pour clore à sa fin), stop (--code N), fenetres / fermer (3 fenêtres de graphiques sans run).

**Prérequis :** zeta_env (numpy, pandas, matplotlib, reportlab) ; monitoring/scripts/zeta_mon_common.py.

**Ce qu'il modifie / produit :** monitoring/runs/test|prod/<id>/ avec meta.json, session.md, etat.json/etat.csv, metrics.csv, figures/, rapport.md/.pdf ; met à jour Monitoring-Runs.md ; lance un superviseur (priorité minimale).

**Comment le lancer :**

```
zeta-mon-exec --nom T-test --type test --csv ~/projet_zeta/calculs/ -- scripts/zeta_run.sh
    5000
zeta-mon-start --nom NOM --type prod [--pid PID]
zeta-mon-stop --code 0
zeta-mon-fenetres / zeta-mon-fermer
```

■ **À savoir :** Fenêtres : auto = oui pour un run de test, non pour un run de production (risque de gel déjà vécu). Wayland : la position des fenêtres peut être ignorée. Intégré à scripts/zeta_run.sh depuis le 17/10/2026 (variable ZETA_MON_TYPE=test|prod, défaut prod).

**Sudo :** non · **Sensible :** oui (lance un run ; ouvre des fenêtres) · **Statut :** testé pour exec (runs T=50 000 et T=100 000 du 17/10/2026), fermer et fenêtres via exec ; start, stop et fenetres autonomes : non testés.

---

## monitoring/scripts/zeta_run_watch.py

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Détecte AUTOMATIQUEMENT qu'un run tourne (processus dont la ligne de commande contient le motif, défaut « compute_zeros »), l'enregistre, et à la fin génère le rapport et met à jour Monitoring-Runs.md. Le CSV des zéros est celui donné par zeta-run-hint, sinon le plus récent sous calculs/.

**Prérequis :** zeta_env (numpy, pandas, matplotlib, reportlab) ; monitoring/scripts/zeta_mon_common.py.

**Ce qu'il modifie / produit :** monitoring/runs/<id>/ ; logs/watch.log.

**Comment le lancer :**

```
lancé par zeta-mon-on (python zeta_run_watch.py --pattern …)
```

■ **À savoir :** Début du run = démarrage du plus ancien processus détecté ; fin = plus aucun processus pendant --confirm sondages consécutifs.

**Sudo :** non · **Sensible :** non · **Statut :** processus actif vérifié le 17/10/2026 ; détection automatique d'un vrai run NON vérifiée (les runs du 17/10 sont passés par le lanceur exec).

---

## monitoring/scripts/zeta_run_report.py

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Rapport de monitoring d'UN run : 4 figures (metrics, zeros_tranches, gue, debit) et commentaires, en MD et PDF (français, ou anglais avec --lang en) ; --index régénère Monitoring-Runs.md.

**Prérequis :** zeta_env (numpy, pandas, matplotlib, reportlab) ; monitoring/scripts/zeta_mon_common.py.

**Ce qu'il modifie / produit :** rapport.md + rapport.pdf (ou report_en.md/.pdf) dans le dossier du run ; interpretation.md est inclus s'il existe.

**Comment le lancer :**

```
zeta-report [dossier_run] [--lang fr|en]
zeta-index
```

■ **À savoir :** Les affirmations sont marquées mesuré / heuristique / hypothèse / conjecture ; le rapport ne remplace pas une validation Turing–Backlund. Trois défauts corrigés le 17/10/2026 (fuseau de --apres, conflit de variables, débit instantané nul affiché comme 0 z/s).

**Sudo :** non · **Sensible :** non · **Statut :** testé (rapports FR et EN générés pour les runs T=50 000 et T=100 000 le 17/10/2026).

---

## monitoring/scripts/zeta_log_metrics.py

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Enregistre toutes les N secondes (défaut 1 s) la température, la fréquence, la charge et la RAM des 5 machines dans monitoring/data/cluster_metrics_AAAAMMJJ.csv. Lecture seule sur les machines distantes.

**Prérequis :** zeta_env (numpy, pandas, matplotlib, reportlab) ; monitoring/scripts/zeta_mon_common.py.

**Ce qu'il modifie / produit :** CSV du jour (une ligne par machine et par seconde).

**Comment le lancer :**

```
zeta-mon-on / zeta-mon-off
python monitoring/scripts/zeta_log_metrics.py --test
```

■ **À savoir :** --test affiche une mesure sans rien écrire. PC4 (OpenBSD) n'a pas de mesure de RAM.

**Sudo :** non · **Sensible :** non · **Statut :** testé (--test : PC1 à PC5 OK le 17/10/2026 ; logger actif).

---

## monitoring/scripts/zeta_plot_live.py

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Suivi graphique en direct d'un run : en haut, zéros par tranche de t vs densité théorique ; en bas, espacement normalisé de chaque zéro (rafraîchi toutes les 2 s, lecture incrémentale du CSV).

**Prérequis :** zeta_env (numpy, pandas, matplotlib, reportlab) ; monitoring/scripts/zeta_mon_common.py.

**Ce qu'il modifie / produit :** fenêtre graphique.

**Comment le lancer :**

```
zeta-live [csv|dossier] [options]
```

■ **À savoir :** Prend le CSV le plus récemment modifié (option --apres pour ignorer les anciens) ; n'affiche rien tant que le CSV n'a pas 3 zéros. Alias marqué ⚠ (fenêtre).

**Sudo :** non · **Sensible :** oui (fenêtre ; à éviter pendant un gros run) · **Statut :** testé (fenêtre confirmée par hprzeta le 17/10/2026 après correction du fuseau de --apres).

---

## monitoring/scripts/zeta_plot_metrics.py

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Historique des 5 machines sur 4 graphiques empilés (température, fréquence, charge, RAM) ; mode direct ou image (--sortie PNG).

**Prérequis :** zeta_env (numpy, pandas, matplotlib, reportlab) ; monitoring/scripts/zeta_mon_common.py.

**Ce qu'il modifie / produit :** fenêtre graphique ou PNG.

**Comment le lancer :**

```
zeta-metrics [minutes] [refresh_s]
python monitoring/scripts/zeta_plot_metrics.py --debut ISO --fin ISO --sortie m.png
```

■ **À savoir :** Une couleur fixe par machine. Alias marqué ⚠ (fenêtre).

**Sudo :** non · **Sensible :** oui (fenêtre) · **Statut :** testé (fenêtre vue par hprzeta le 17/10/2026) ; mode --sortie : non testé ici.

---

## monitoring/scripts/zeta_gue_hist.py

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Histogramme des espacements normalisés des zéros comparé à la loi GUE (surmise de Wigner) et à Poisson. La concordance avec GUE est la conjecture de Montgomery–Odlyzko (non démontrée).

**Prérequis :** zeta_env (numpy, pandas, matplotlib, reportlab) ; monitoring/scripts/zeta_mon_common.py.

**Ce qu'il modifie / produit :** fenêtre graphique ou PNG (--sortie).

**Comment le lancer :**

```
zeta-gue [csv|dossier] [--live]
```

■ **À savoir :** En mode --live il attend --min-zeros (3000 par défaut) puis se rafraîchit toutes les 30 s. Alias marqué ⚠ (fenêtre).

**Sudo :** non · **Sensible :** oui (fenêtre) · **Statut :** testé (fenêtre confirmée par hprzeta le 17/10/2026).

---

## monitoring/scripts/zeta_mon_common.py

Machine : PC1 · Catégorie : Monitoring

**À quoi ça sert :** Chemins et utilitaires communs du monitoring (importé par les autres scripts, non lancé directement) : dossiers runs/data, résolution du CSV d'un run, fenêtres, indice de run.

**Prérequis :** zeta_env (numpy, pandas, matplotlib, reportlab) ; monitoring/scripts/zeta_mon_common.py.

**Ce qu'il modifie / produit :** aucune sortie propre.

**Comment le lancer :**

```
(module importé, pas de commande)
```

■ **À savoir :** À ne pas lancer seul.

**Sudo :** non · **Sensible :** non · **Statut :** testé indirectement (utilisé par tous les scripts du monitoring le 17/10/2026).

# 8d. V17, tests et outils de consolidation

Code de la phase V17 (voir Handoff). Lancement toujours depuis ~/projet_zeta avec `zeta_env/bin/python3 -I` (ou -B -I pour les tests).

## `scripts_v17/consolider_v17.py`

Machine : PC1 · Catégorie : V17 / tests

**À quoi ça sert :** Consolidation NON DESTRUCTIVE des candidats-zéros : chaque groupe examiné reçoit un statut DUPLICATE_CONFIRMED / DISTINCT / AMBIGUOUS ; la proximité numérique n'est jamais une preuve d'identité. Remplace à titre expérimental dedupliquer() de parallel_scanner.py sans le modifier.

**Prérequis :** zeta_env (Python 3.12).

**Ce qu'il modifie / produit :** module (fonction consolider()) ; sorties dans consolidation_v17/ (consolide.csv, decisions.jsonl, ambigus.csv, invalides.csv, manifeste.json) quand appelé par compute_zeros_v17.py.

**Comment le lancer :**

```
(module importé ; voir rejeu_t1b.py)
```

■ **À savoir :** Une paire AMBIGUOUS reste conservée telle quelle (décision 15/10/2026).

**Sudo :** non · **Sensible :** non · **Statut :** testé (tests_v17/test_consolider_v17.py : 16 PASS le 15/10/2026 ; rejeu T1b : 185 821 → 185 814).

## `scripts_v17/rejeu_t1b.py`

Machine : PC1 · Catégorie : V17 / tests

**À quoi ça sert :** Rejeu de la consolidation V17 sur les données BRUTES de T1b (lecture seule) : aucun zéro recalculé ; compare à l'ancienne dedupliquer(0.01).

**Prérequis :** zeta_env (Python 3.12).

**Ce qu'il modifie / produit :** écrit dans un NOUVEAU dossier (refuse d'écraser).

**Comment le lancer :**

```
zeta_env/bin/python3 -I scripts_v17/rejeu_t1b.py <dossier_sortie>
```

■ **À savoir :** Lit calculs/v16_T4900000_4986000_T1b/checkpoints/worker_*.csv.

**Sudo :** non · **Sensible :** non · **Statut :** testé (rejeu du 15/10/2026 ; checkpoints v16/v17 identiques bit à bit le 16/10/2026).

## `scripts_v17/zsign_paires.py`

Machine : PC1 · Catégorie : V17 / tests

**À quoi ça sert :** Test Z-sign CIBLÉ sur les 4 paires rapprochées de T1b : rejoue le scan, raffine, évalue Z avec mpmath (dps 30 et 50) ; corroboration numérique, PAS une certification.

**Prérequis :** zeta_env (Python 3.12).

**Ce qu'il modifie / produit :** écrit dans resultats_v17/ (non versionné).

**Comment le lancer :**

```
zeta_env/bin/python3 -I scripts_v17/zsign_paires.py <dossier_sortie>
```

■ **À savoir :** Un changement de signe étaye l'existence d'au moins un zéro ; il ne prouve pas l'unicité.

**Sudo :** non · **Sensible :** non · **Statut :** testé (15/10/2026 : 2 changements de signe par paire, CORROBORATED_NUMERICALLY).

### tests_v17/test_consolider_v17.py

Machine : PC1 · Catégorie : V17 / tests

**À quoi ça sert :** Tests unitaires D1-D13 de consolider_v17 sur données SYNTHÉTIQUES (aucun calcul de zêta). D1-D13 viennent du rapport d'audit du 15/10/2026, pas du dépôt.

**Prérequis :** zeta_env (Python 3.12).

**Ce qu'il modifie / produit :** rien.

**Comment le lancer :**

```
zeta_env/bin/python3 -B -I tests_v17/test_consolider_v17.py
```

■ **À savoir :** Premier passage : 4 FAIL sur 14 (dont D5, défaut réel de la règle, corrigé par R-RAYON).

**Sudo :** non · **Sensible :** non · **Statut :** testé (16/16 PASS le 15/10/2026).

---

### tests_v17/test_compute_zeros_v17.py

Machine : PC1 · Catégorie : V17 / tests

**À quoi ça sert :** Tests ciblés de compute_zeros_v17.py (revue du 16/10/2026) : D1, D2, D6, avec de petits calculs réels (T ≤ 400) et des échecs d'E/S simulés (unittest.mock).

**Prérequis :** zeta_env (Python 3.12).

**Ce qu'il modifie / produit :** rien (pas de disque plein réel).

**Comment le lancer :**

```
zeta_env/bin/python3 -B -I tests_v17/test_compute_zeros_v17.py
```

■ **À savoir :** —

**Sudo :** non · **Sensible :** non · **Statut :** testé (9 OK le 16/10/2026).

---

### tests_turbo/test_zeta_turbo.py

Machine : PC1 · Catégorie : V17 / tests

**À quoi ça sert :** Tests du cycle de vie du turbo (zeta_turbo_on/off/status.sh, zeta_session_turbo.sh) sur un arbre sysfs/proc FACTICE et un faux systemctl : aucune modification du système réel.

**Prérequis :** zeta_env (Python 3.12).

**Ce qu'il modifie / produit :** dossier temporaire uniquement.

**Comment le lancer :**

```
zeta_env/bin/python3 -B -I tests_turbo/test_zeta_turbo.py
```

■ **À savoir :** 9 mutations détectées lors de la validation (16/10/2026).

**Sudo :** non (aucun sudo réel) · **Sensible :** non · **Statut :** testé (34 OK le 16/10/2026).

## 8e. Code de calcul et utilitaires de src/ (inventaire du 17/10/2026)

---

Le CLAUDE.md du projet liste les versions v2, v4 et v5 de compute_zeros comme « NE PAS MODIFIER » ; compute_zeros_v16.py est la version lancée par scripts/zeta_run.sh, compute_zeros_v17.py est la version durcie en cours de validation. Sudo : non pour tous ces fichiers. Sensible : non (les compute_zeros_v* lancent de gros calculs : à passer par zeta_run.sh, jamais directement).

## src/calculs/optimisation/ (calcul des zéros)

| Fichier | Rôle (d'après l'en-tête) | Statut |
|---|---|---|
| calculs/optimisation/__init__.py | fichier vide (0 octet), marqueur de paquet Python | non testé dans cette session |
| calculs/optimisation/arb_wrapper.py | Z(t) via Arb/FLINT arb_fpwrap_cdouble_hardy_z | non testé dans cette session |
| calculs/optimisation/benchmark_15min.py | Comparaison de performance sur 15 minutes | non testé dans cette session |
| calculs/optimisation/benchmark_methodes.py | Benchmark illinois_refine_bench sur 500 zéros T=100k v7 | non testé dans cette session |
| calculs/optimisation/benchmark_v8.py | Benchmark Option A (W=8 workers) + Option B (prec phase 2) | non testé dans cette session |
| calculs/optimisation/chrono_phases.py | Profileur léger pour localiser le goulot du pipeline v4.1. | non testé dans cette session |
| calculs/optimisation/compute_zeros_v10.py | Phase C v10 : W=8 workers forcés (vs W=4 capé en v9) | non testé dans cette session |
| calculs/optimisation/compute_zeros_v12.py | Phase C v12 : Illinois C pur via arb_fpwrap_cdouble_hardy_z | non testé dans cette session |
| calculs/optimisation/compute_zeros_v13.py | Phase C v13 : seuil petit-t abaissé à 20 | non testé dans cette session |
| calculs/optimisation/compute_zeros_v14.py | Phase C v14 : Illinois 1 Newton Z_arb + cache RS | non testé dans cette session |
| calculs/optimisation/compute_zeros_v15.py | Phase C v15 : Phase 2 adaptative (SEUIL_1NEWTON=20k) | non testé dans cette session |
| calculs/optimisation/compute_zeros_v15_test_lowprec.py | VALIDATION EXPÉRIMENTALE, PAS UN SCRIPT DE PRODUCTION | non testé dans cette session |
| calculs/optimisation/compute_zeros_v16.py | Phase C v16 : Z_arb à précision fixe (acb_dirichlet_hardy_z) | utilisé pour les runs de test du 17/10/2026 (T=50 000 et 100 000) |
| calculs/optimisation/compute_zeros_v17.py | copie de v16 durcie (v17-a à v17-d) : traçabilité, checkpoints, consolidation | testé (tests_v17 ; rejeu T1b) ; pas de run T=5M |
| calculs/optimisation/compute_zeros_v3.py | Version optimisée (intègre toutes les corrections) | non testé dans cette session |
| calculs/optimisation/compute_zeros_v4.py | Phase C : Illinois C/libmpfr + mpmath.siegelz | non testé dans cette session |
| calculs/optimisation/compute_zeros_v4_1.py | Phase C : Illinois C + Z_batch + parallèle | non testé dans cette session |
| calculs/optimisation/compute_zeros_v5.py | Phase C Voie B : illinois_mpfr_cb + mpmath.siegelz | non testé dans cette session |
| calculs/optimisation/compute_zeros_v6.py | scan_arb.c (Z_double C) + STEP=0.010 + N(T) bissection | non testé dans cette session |
| calculs/optimisation/compute_zeros_v7.py | illinois_refine_adaptive prec_fast=64 bits + prec_full=80 bits | non testé dans cette session |
| calculs/optimisation/compute_zeros_v8.py | Phase C v8 : prec_fast=64 bits + prec_full=80 bits (optimal) | non testé dans cette session |
| calculs/optimisation/compute_zeros_v9.py | Phase C v9 : Brent C/mpfr 2-phases (prec_fast=64, prec_full=80) | non testé dans cette session |
| calculs/optimisation/parallel_scanner.py | Calcul parallèle des zéros par partitionnement | non testé dans cette session |
| calculs/optimisation/patch_workprec.py | applique workprec(50) sur le bloc mpmath_petit_t | non testé dans cette session |
| calculs/optimisation/riemann_siegel_batch.py | Z(t) vectorisé CPU (numpy) ou GPU (CuPy) | non testé dans cette session |
| calculs/optimisation/scan_arb_wrapper.py | Interface Python vers scan_zeros_arb (scan_arb.so) | non testé dans cette session |
| calculs/optimisation/test_ab_scanarb_20260906.py | Test A/B ciblé — piste "bruit de phase Z_double" sur les 96 zéros manquants v13. | non testé dans cette session |

| Fichier | Rôle (d'après l'en-tête) | Statut |
|---|---|---|
| calculs/optimisation/test_brent.py | Validation brent_refine_adaptive vs illinois_refine_bench | non testé dans cette session |
| calculs/optimisation/test_v4_1_T300.py | Point d'arrêt N°2 : run T=300, comparaison vs v5 validé | non testé dans cette session |
| calculs/optimisation/test_zbatch_validation.py | Point d'arrêt N°1 (v2) : Z_vect_correct vs mpmath.siegelz | non testé dans cette session |
| calculs/optimisation/theta_rapide.py | Calcul rapide de la fonction θ(t) de Riemann-Siegel | non testé dans cette session |
| calculs/optimisation/turing_validation.py | Validation rigoureuse par le critère de Turing-Backlund | non testé dans cette session |
| calculs/optimisation/validate_v4_1.py | Vérifications A et B pour compute_zeros_v4_1.py | non testé dans cette session |

## src/calculs/optimisation/c_modules/ (Phase C : libmpfr / Arb)

Compilation par le Makefile du dossier (voir son CLAUDE.md). diagnostic_gamma_c_vs_mpmath.sh est documenté plus haut (§8).

| Fichier | Rôle (d'après l'en-tête) | Statut |
|---|---|---|
| calculs/optimisation/c_modules/benchmark_illinois.py | illinois_refine C Option B vs mpmath sur 100 zéros à t ∈ [500, 1100] | non testé dans cette session |
| calculs/optimisation/c_modules/brent_mpfr.c | Méthode de Brent 2-phases via libmpfr ═══════ Algorithme de Van Wijngaarden-Dekker-Brent : I | non testé dans cette session |
| calculs/optimisation/c_modules/compare_arb_vs_mpfr.py | comparaison directe illinois_refine_arb (Arb/FLINT) | non testé dans cette session |
| calculs/optimisation/c_modules/diagnostic_gamma_c_vs_mpmath.sh | diagnostic_gamma_c_vs_mpmath.sh | non testé dans cette session |
| calculs/optimisation/c_modules/gen_brackets.py | génère des brackets [a,b,fa,fb] représentatifs pour profiler | non testé dans cette session |
| calculs/optimisation/c_modules/illinois_arb.c | affinage Illinois via acb_dirichlet_hardy_z (précision fixe) v16 (2026-08-08) — remplacement de Z_arb vs v15 : arb_fpwrap_cdouble_hardy_z(fl | non testé dans cette session |
| calculs/optimisation/c_modules/illinois_arb_lowprec.c | variante expérimentale de illinois_refine_arb Différence avec illinois_arb.c (production, NE PAS MODIFIER) : Phase 2 utilise acb_dirichlet_h | non testé dans cette session |
| calculs/optimisation/c_modules/illinois_mpfr.c | affinage Illinois de la méthode de la sécante en libmpfr Précision : PREC = 170 bits (≈ 51 décimales) — phase d'affinage uniquement Algorith | non testé dans cette session |
| calculs/optimisation/c_modules/illinois_pyZ.py | Voie B : wrappers Python pour illinois_mpfr.so | non testé dans cette session |
| calculs/optimisation/c_modules/perf_harness.py | charge illinois_arb.so via ctypes et boucle sur des brackets | non testé dans cette session |
| calculs/optimisation/c_modules/scan_arb.c | Détection C pure des changements de signe de Z(t) via formule Riemann-Siegel C0+C1 en double précision v14 (2026-07-04) — 2 optimisations vs | non testé dans cette session |
| calculs/optimisation/c_modules/test_illinois.py | validation de illinois_mpfr (C/libmpfr) | non testé dans cette session |
| calculs/optimisation/c_modules/test_illinois_arb.py | validation de illinois_refine_arb sur les 20 premiers zéros | non testé dans cette session |
| calculs/optimisation/c_modules/test_illinois_v5.py | validation Voie B : illinois_mpfr_cb + callback mpmath.siegelz | non testé dans cette session |
| calculs/optimisation/c_modules/test_z.c | petit programme C (d'après le code) : affiche pour une série de t les valeurs Z_double, Z_mpfr et l'écart absolu (err) | non testé dans cette session |

| Fichier | Rôle (d'après l'en-tête) | Statut |
|---|---|---|
| calculs/optimisation/c_modules/z_function.c | theta(t) Stirling + Z(t) Riemann-Siegel + correction C0+C1 Usage : detection des changements de signe (balayage) — doit etre IDENTIQUE a Z_m | non testé dans cette session |

## src/calculs/optimisation/no batch_version/ (anciennes versions)

| Fichier | Rôle (d'après l'en-tête) | Statut |
|---|---|---|
| calculs/optimisation/no batch_version/parallel_scanner.py | Calcul parallèle des zéros par partitionnement | non testé dans cette session |
| calculs/optimisation/no batch_version/riemann_siegel.py | Formule de Riemann-Siegel pour Z(t) | non testé dans cette session |

## Autres fichiers de src/

| Fichier | Rôle (d'après l'en-tête) | Statut |
|---|---|---|
| __init__.py | fichier vide (0 octet), marqueur de paquet Python | non testé dans cette session |
| benchmark/affinage_arb.py | Benchmark : arb_fpwrap_cdouble_hardy_z (Arb/FLINT 3) vs mpmath.siegelz | non testé dans cette session |
| ia/__init__.py | fichier vide (0 octet), marqueur de paquet Python | non testé dans cette session |
| main.py | Point d'entrée principal du projet Zêta | non testé dans cette session |
| monitoring/logger_config1.py | Configuration centralisée des logs | non testé dans cette session |
| utils/__init__.py | fichier vide (0 octet), marqueur de paquet Python | non testé dans cette session |
| utils/transform_svg_to_png.py | convertit un SVG en PNG avec cairosvg (d'après le code) ; le chemin du SVG est écrit en dur dans le script | non testé dans cette session |
| visualisation/plots1.py | Génère les graphiques de distribution | non testé dans cette session |

## cours/ et pdf/clone/ (génération de documents)

| Fichier | Rôle (d'après l'en-tête) | Statut |
|---|---|---|
| cours/generate_pdf.py | Génère Cours_Zeta_HR_complet.pdf à partir de Cours_Zeta_HR_complet.md (ReportLab). | non testé dans cette session |
| cours/make_wiki_page.py | Convertit Cours_Zeta_HR_complet.md en page wiki unique | non testé dans cette session |
| pdf/clone/generate_pdf.py | Régénère rapport_clonage_zeta_enrichi.pdf depuis rapport_clonage_zeta.md via WeasyPrint. | non testé dans cette session |
| pdf/clone/generate_pdf_maj.py | Génère MAJ_clone_ubuntu_20260824.pdf depuis MAJ_clone_ubuntu_20260824.md via WeasyPrint. | non testé dans cette session |

### Archives et essais (regroupés : documentés par le seul nom)

| Dossier | Nb | Fichiers |
|---|---|---|
| docs/archive/python_obsoletes/ | 16 | calcul_zeta.py, compute_zeros_test_error1.py, compute_zeros_test_error1_corr1.py, compute_zeros_test_error2.py, compute_zeros_v1.py, compute_zeros_v2.py, config_loader.py, config_loader1.py, cpu_temp_monitor.py, demo_complete.py, ia_zeta.py, logger_config.py, plots.py, prng_zeta_aleatoire.py, verification.py, zeros_finder.py |
| docs/archive/scripts_shell/ | 4 | install_zeta_complete.sh, monitor2.sh, run_computation.sh, setup_ollama_final.sh |
| src/tests/cours/ | 26 | n0_animation_cerle.py, n0_animation_eitheta_t_cerle.py, n0_complexe_exemple.py, n1_animation_exploration_zeta.py, n1_animation_approxim_taylor.py, n1_animation_convergence_zeta_2.py, n1_animation_suite.py, n1_animation_vitesse_convergence_zeta_2.py, n2_ Prolongement_série.py, n2_animation_intégration_ numérique_contour.py, n2_animation_visualisation_gama.py, n2_animation_vérif_conditions_Cauchy-Riemann.py, n2_visualisation du prolongement.py, n2_visualisation_zeta_sur_tout_C.py, n3_Produit_eulériene.py, n3_visualisation_produit_eulérien_et_convergence.py, n3_visualisation_transformée_Mellin.py, n3_ visualisation_vérification_equation_fonctionnelle.py, n4_visualisation_calcul _et_tracé_ de_Z.py, n4_visualisation_densité_et_répartition_ zéros.py, n4_visualisation_formule de Riemann-Siegel_recherche_de_ zéros.py, n4_visualisation_visualisation_des_zéros.py, n5_visualisation_PRNG_ test_uniformité.py, n5_visualisation_calcul_fonctions_L .py, n5_visualisation_formule _explicite_reconstruction_de_Phy.py, n5_visualisation_statistiques_ GUE_vs_zéros_zeta.py |
| src/tests/kimi_dev/src/ | 3 | parallel_scanner.py, riemann_siegel.py, turing_validation.py |
| src/tests/ (autres) | 1 | factorielle_ gamma_ zeta .py |

Ces fichiers sont des archives ou des essais : aucune description individuelle n'est donnée. Non testés dans cette session.

# 9. Scénarios courants — « je veux… »

| Je veux… | Commande (sur PC1) |
|---|---|
| lancer un calcul simple | `zeta-run 10000` |
| lancer un gros calcul distribué PC1+PC2 | `zeta-distribute 500000` |
| suivre un run en cours | `zeta-progress-tmux` ou `zeta-progress` |
| surveiller l'état des 5 machines | `zeta-monitor` |
| voir les températures des 5 machines | `zeta-temp` |
| ouvrir la session cluster | `zeta-cluster` |
| synchroniser le code vers PC2 | `bash scripts/zeta_sync_pc2.sh` |
| inventorier le cluster (lecture seule) | `zeta-inventaire` |

| Je veux… | Commande (sur PC1) |
|---|---|
| vérifier / monter le VPN | `wg-auto` |
| voir les MàJ dispo sur une machine | `zeta-update --only "PC2"` |
| mettre à jour une machine | `zeta-update --apply --only "PC2"` |
| vérifier que le backup Proton va bien | `zeta-proton` |
| réparer le backup Proton (token expiré) | `(sur PC3) bash ~/fix_proton.sh` |
| vérifier le pipeline de backup | `zeta-backup-status` |
| sauvegarder ce qui est hors git sur Proton | `zeta-backup-horsgit` |
| cloner PC1 sur le disque Toshiba | `zeta-clone` |
| gérer le SSD du RAG (monter, démonter…) | `zeta-rag` |
| interroger le RAG | `python scripts/rag_query.py "ma question"` |
| convertir un PDF de cours en .md | `./scripts/pdf_to_md_rapide.sh ~/projet_zeta/pdf/X` |
| générer un PDF depuis un .md du wiki | `./scripts/build_pdf_riemann.sh source.md` |

# 10. Aide-mémoire des alias (PC1)

| Alias | Équivaut à |
|---|---|
| `zeta` | se placer dans ~/projet_zeta et activer le venv zeta_env |
| `zeta-proj` | cd ~/projet_zeta/ |
| `zeta-docs` | cd ~/projet_zeta/docs |
| `zeta-data` | cd /mnt/data |
| `zeta-logs` | tail -f /mnt/data/logs/mon_projet.log |
| `zeta-python` | venv + cd ~/projet_zeta/src/calculs |
| `zeta-code` | venv + ouvre le projet dans VS Code |
| `zeta-spyder` | venv + lance Spyder |
| `zeta-notebook` | venv + Jupyter Notebook dans notebooks/ |
| `zeta-jupyter` | venv + JupyterLab dans notebooks/ |
| `zeta-tmux` | tmux attach -t zeta (ou nouvelle session « zeta ») |
| `zeta-run` | ~/projet_zeta/scripts/zeta_run.sh |
| `zeta-turbo-on` | sudo scripts/zeta_turbo_on.sh |
| `zeta-turbo-off` | sudo scripts/zeta_turbo_off.sh |
| `zeta-distribute` | bash scripts/zeta_distribute_run.sh |
| `zeta-progress` | venv + python scripts/zeta_run_progress.py |
| `zeta-progress-tmux` | tmux attach -t zeta-progress |
| `zeta-log` | suit le dernier log distribute_T*_nohup_*.log |
| `zeta-pid` | affiche le pid du run distribué (/tmp/zeta_distribute.pid) ou « Aucun run actif » |
| `zeta-cluster` | scripts/zeta_tmux.sh |
| `zeta-monitor` | python3 scripts/zeta_monitor.py |
| `zeta-temp` | venv + python3 scripts/zeta_temp_monitor.py |
| `zeta-webmonitor` | venv + python3 scripts/zeta_webmonitor.py : **NON FONCTIONNEL**, ce script n'existe pas encore (à créer) ; rôle prévu : centraliser les scripts de monitoring pour tout lancer d'un coup (voir §10b) |
| `zeta-update` | bash scripts/zeta_update.sh |
| `zeta-inventaire` | bash scripts/zeta_inventaire.sh |
| `zeta-proton` | bash scripts/zeta_proton_status.sh |
| `zeta-backup-status` | venv + python scripts/zeta_backup_status.py |
| `zeta-backup-horsgit` | scripts/zeta_backup_horsgit.sh |
| `zeta-clone` | sudo bash scripts/zeta_backup_toshiba.sh |

| Alias | Équivaut à |
|---|---|
| zeta-rag | bash scripts/zeta_rag.sh |
| zeta-ecran-off | gnome-extensions disable (extension hanabi, fond d'écran animé) |
| zeta-ecran-on | gnome-extensions enable (extension hanabi) |
| wg-auto | bash scripts/wg_auto.sh |

# 10b. Alias et fonctions : compléments (n° du menu zeta-script)

Menu zeta-script au 17/10/2026 : 58 entrées (n° 1 à 36 = alias de ~/.bashrc, n° 37 à 51 = fonctions du monitoring, n° 52 à 58 = couleurs de terminal). Les alias n° 1 à 32 sont dans le tableau ci-dessus. ⚠ = marqué sensible dans le menu.

| N° | Alias | Principe / équivaut à | Usage | Sudo | Sensible | Statut |
|---|---|---|---|---|---|---|
| 33 | zeta-capture-conv | enregistre la session du terminal (script capture.sh) ; à lancer AVANT claude | zeta-capture-conv | non | non | non testé dans cette session |
| 34 | zeta-convert-jsontomd | python3 -I scripts/zeta_jsonl_to_md.py : session Claude Code → Markdown masqué | zeta-convert-jsontomd [SESSION] [-o DOSSIER] [--liste] | non | non | non testé dans cette session |
| 35 | zeta-export-claudeai | bash scripts/zeta_export_claudeai.sh : état du projet masqué pour claude.ai | zeta-export-claudeai | non | non | non testé dans cette session |
| 36 | zeta-script | python3 -I scripts/zeta_script.py : menu de ces alias | zeta-script | non | non | testé (17/10/2026) |
| 37 | zeta-mon-on | arrête puis relance le logger (1 s, 5 machines) et le détecteur de runs, puis affiche le statut | zeta-mon-on [options du logger] | non | non | testé (17/10/2026) ; à refaire après chaque redémarrage de PC1 |
| 38 | zeta-mon-off | pkill du logger et du détecteur de runs | zeta-mon-off | non | non | non testé ici |
| 39 | zeta-mon-status | processus actifs, 5 dernières mesures, derniers runs, journal du watcher | zeta-mon-status | non | non | testé (17/10/2026) |
| 40 | zeta-live | zeta_plot_live.py : zéros par tranche en direct | zeta-live [csv ou dossier] [options] | non | ⚠ fenêtre | testé (17/10/2026) |
| 41 | zeta-metrics | zeta_plot_metrics.py : T°/fréquence/load/RAM des 5 machines en direct | zeta-metrics [minutes=15] [refresh_s=2] | non | ⚠ fenêtre | testé (17/10/2026) |
| 42 | zeta-gue | zeta_gue_hist.py : espacements vs GUE | zeta-gue [csv ou dossier] [--live] | non | ⚠ fenêtre | testé (17/10/2026) |
| 43 | zeta-mon-exec | enveloppe une commande sous monitoring (rapport MD+PDF à la fin) | zeta-mon-exec --nom N --type test ou prod [--csv X] [--fenetres auto, oui ou non] -- commande | non | non (lance pourtant un run) | testé (runs du 17/10/2026, via zeta_run.sh) |
| 44 | zeta-mon-start | démarre le suivi d'un run lancé à côté | zeta-mon-start --nom N --type test ou prod [--pid PID] | non | non | non testé ici |
| 45 | zeta-mon-stop | clôt le suivi d'un run (génère le rapport) ; inutile avec exec | zeta-mon-stop --code N [--attendre] | non | non | non testé ici (sans run actif : « aucun run suivi ») |

| N° | Alias | Principe / équivaut à | Usage | Sudo | Sensible | Statut |
|---|---|---|---|---|---|---|
| 46 | `zeta-mon-fenetres` | ouvre les 3 fenêtres de graphiques rangées, sans run | zeta-mon-fenetres [--csv X] | non | ⚠ fenêtre | non testé seul ici |
| 47 | `zeta-mon-fermer` | ferme les fenêtres ouvertes par le lanceur (PID dans runs/.fenetres.json) ; ne ferme pas celles ouvertes à la main | zeta-mon-fermer | non | non | testé (3 fenêtres fermées le 17/10/2026) |
| 48 | `zeta-run-hint` | AVANT un run : enregistre le nom et le CSV de sortie attendus (runs/.hint) | zeta-run-hint NOM [CSV ou dossier] | non | non | non testé ici |
| 49 | `zeta-report` | régénère le rapport d'un run (--lang en = version définitive) | zeta-report [dossier_run] [--lang en] | non | non | testé (17/10/2026) |
| 50 | `zeta-index` | régénère Monitoring-Runs.md (liste des runs) | zeta-index | non | non | testé (via install.sh, 17/10/2026) |
| 51 | `zeta-runs` | ls -1t sur monitoring/runs (20 premiers résultats) ; depuis la v2 il liste les sous-dossiers (test, prod, historique), pas les runs eux-mêmes | zeta-runs | non | non | testé (sortie : test, historique) |

## Couleurs de terminal par machine (fichier ~/zeta_colors.sh), n° 52 à 58 du menu

Chargé par ~/.bashrc (ligne 147). Ces 7 commandes ont été ajoutées au menu zeta-script le 17/10/2026 (n° 52 à 58, dans l'ordre : zeta_set_bg, zeta_reset_bg, zeta-reset, zeta-hp, zeta-acer, zeta-del, zeta-color-test). Elles changent la couleur de fond du terminal le temps d'une connexion SSH.

| N° | Commande | Principe | Sudo | Sensible | Statut |
|---|---|---|---|---|---|
| 52 | `zeta_set_bg` | fonction de base : zeta_set_bg "#RRGGBB" ["#RRGGBB"] = fond [et texte] (séquences OSC 11 / 10) | non | non | non testé dans cette session |
| 53 | `zeta_reset_bg` | fonction de base : remet fond et texte par défaut (OSC 110 / 111) | non | non | lancé via le menu le 17/10/2026 (séquences émises ; effet visuel non vérifié) |
| 54 | `zeta-reset` | alias de zeta_reset_bg : remet le fond et le texte par défaut du profil (utile si une session a planté) | non | non | non testé dans cette session |
| 55 | `zeta-hp` | fond jaune, texte noir, puis ssh zeta-hp (PC2) ; remet les couleurs au retour | non | non | non testé dans cette session |
| 56 | `zeta-acer` | fond bleu clair, texte noir, puis ssh zeta-acer (PC3) ; remet les couleurs au retour | non | non | non testé dans cette session |
| 57 | `zeta-del` | fond noir, texte vert (style bastion), puis ssh zeta-del (PC4) ; remet les couleurs au retour | non | non | non testé dans cette session |
| 58 | `zeta-color-test` | prévisualise les 3 couleurs sans SSH (touche Entrée pour passer à la suivante) | non | non | non testé dans cette session |

Note : si zeta-hp, zeta-acer ou zeta-del existent aussi comme ALIAS dans ~/.bashrc, l'alias l'emporte sur la fonction (avertissement de l'en-tête de zeta_colors.sh).

## zeta-webmonitor (n° 7) : cible absente

L'alias pointe vers scripts/zeta_webmonitor.py, qui n'existe pas. **Rôle prévu (décision hprzeta du 17/10/2026)** : ce script devra centraliser tous les scripts de monitoring pour tout lancer d'un coup. Le libellé actuel du menu (« Moniteur web du cluster ») est à harmoniser avec ce rôle. Rien n'a été créé. Sudo : à préciser · Sensible : à préciser · Statut : non créé.

*Mis à jour : 2026-10-17 — 1468 lignes*